

O módulo *Keyboard Reader* é constituído por três blocos principais: *i*) o descodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*), de acordo com o diagrama representado na Figura 1. Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

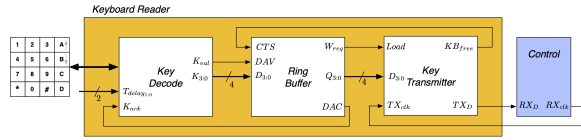
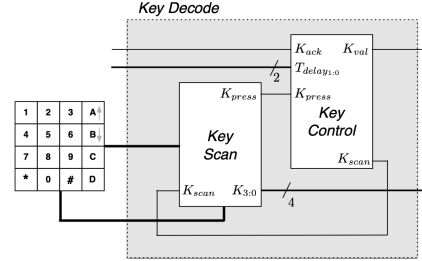
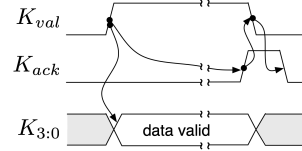


Figura 1: Diagrama de blocos do módulo *Keyboard Reader*



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 2: Bloco *Key Decode*

1 Key Decode

O bloco *Key Decode* implementa um descodificador de um teclado matricial 4x4 por hardware, sendo constituído por três sub-blocos: *i*) um teclado matricial de 4x4; *ii*) o bloco *Key Scan*, responsável pelo varrimento do teclado; e *iii*) o bloco *Key Control*, que realiza o controlo do varrimento e o controlo de fluxo, conforme o diagrama de blocos representado na Figura 2a.

O controlo de fluxo de saída do bloco *Key Decode* (para o módulo *Key Buffer*), define que o sinal K_{val} é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no barramento $K_{3:0}$. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal K_{ack} for ativado e a tecla premida for libertada. O diagrama temporal do controlo de fluxo está representado na Figura 2b.

1.1 Key Scan

O bloco *Key Scan* foi implementado de acordo com o diagrama de blocos representado na Figura 3.

Para a implementação do *Key Scan*, foi utilizada a versão 1, conforme apresentado nos slides compar ativos (página 28):contentReference[oaicite:3]index=3. A escolha desta versão deve-se a vários fatores: • Simplicidade de implementação: a versão 1 apresenta uma arquitetura

mais direta, facilitando a compreensão e desenvolvimento inicial do sistema; • Facilidade de depuração: devido à separação clara entre *Key Scan* e *Key Control*, torna-se mais simples identificar erros durante a simulação; • Adequação ao contexto do projeto: sendoumsistemaacadémico e modular (*Keyboard Reader*), a versão 1 cumpre todos os requisitos funcionais sem necessidade de otimizações adicionais; • Compatibilidade com os restantes blocos: a interface com o *Ring Buffer* e o *Key Transmitter* é garantida de forma simples através do protocolo K_{val}/K_{ack} .

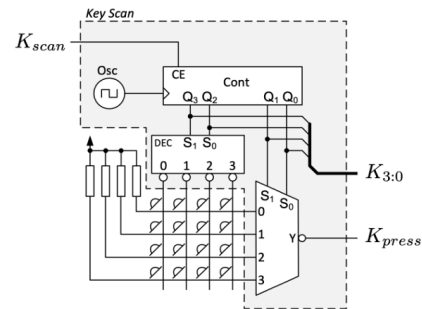


Figura 3: Diagrama de blocos do módulo *Key Scan*

1.2 Key Control

O bloco *Key Control* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 4.

O bloco Key Control implementa a lógica de controlo do sistema, sendo responsável por:

- Controlar o sinal de varrimento (Kscan);
- Gerir a validação da tecla premida (Kval);
- Controlar a inicialização e limpeza do temporizador (Tclear);
- Implementar o protocolo de handshake com o consumidor através do sinal Kack.
- Gerir a lógica de repetição automática da tecla premida por temporização (Repeat e Texpired).

De acordo com o diagrama apresentado nos slides (Figura do Key Decode e diagrama temporal, página 5), o funcionamento segue os seguintes passos: 1. Estado de Scan: O sistema inicia o varrimento do teclado (Kscan ativo) e permanece neste estado até Key Scan detetar uma tecla (Kpress = 1) e segue para o proximo estado(caso contrario permanece);

2. Estado Delivery: O varrimento é interrompido e são ativados simultaneamente os sinais Kval (indicando ao consumidor que os dados da tecla são válidos) e Tclear (que reinicia o temporizador). O sistema aguarda que o sinal Kack seja ativo indicando que os dados em K3:0 são válidos;

3. Estado NotAcknowledge: Após o sinal de Acknowledge (Kack verdadeiro), o sistema transita para um estado intermédio onde aguarda que o sinal de validação deixe de estar ativo (Kack passa a falso), garantindo a sincronização do protocolo;

4. Estado KeyRelease: O sistema avalia se a tecla deve ser repetida continuamente (Repeat).

Se Repeat for falso (a tecla foi libertada ou não há ordem de repetição), o fluxo regressa imediatamente ao estado inicial de varrimento (Scan);

Se Repeat for verdadeiro, o sistema monitoriza o temporizador através do sinal Texpired. Enquanto o tempo não expirar (Texpired falso), permanece neste estado. Assim que o tempo limite é

atingido (Texpired verdadeiro), o fluxo regressa ao segundo estado de Delivery para revalidar e reenviar o código da tecla.

Este mecanismo garante que: • O sistema diferencia corretamente uma pressão única de uma tecla pressionada continuamente; • É integrada de forma automática a funcionalidade de repetição de tecla por timeout (temporização) sem necessitar de varrer novamente.

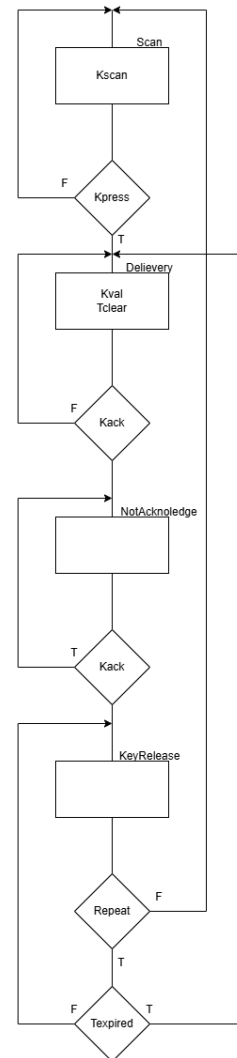


Figura 4: Máquina de estados do *Key Control*

A descrição hardware do bloco *Key Decode* em *VHDL* encontra-se no Anexo ??.

2 Ring Buffer

O bloco *Ring Buffer* implementa uma estrutura de dados para armazenamento de teclas com disciplina FIFO (*First In First Out*), com capacidade de armazenar até dezasseis (16) palavras de quatro (4) bits.

A escrita de dados no *Ring Buffer* inicia-se com a ativação do sinal *DAV* (*Data Available*) pelo sistema produtor, neste caso pelo *Key Decode*, indicando que tem dados para serem armazenados. Logo que tenha disponibilidade para armazenar informação, o *Ring Buffer* escreve os dados $D_{0:3}$ em memória. Concluída a escrita em memória ativa o sinal *DAC* (*Data Accepted*) para informar o sistema produtor que os dados foram aceites. O sistema produtor mantém o sinal *DAV* ativo até que *DAC* seja ativado. O *Ring Buffer* só desativa *DAC* depois de *DAV* ter sido desativado.

A implementação do *Ring Buffer* é baseada numa memória RAM (*Random Access Memory*). O endereço de escrita/leitura, selecionado por *put*(*get*), definido pelo bloco *Memory Address Control* (MAC) composto por dois registos, que contém o endereço de escrita e leitura, designados por *putIndex* e *getIndex* respetivamente. O MAC suporta assim ações de *incPut* e *incGet*, gerando informação se a estrutura de dados está cheia (*Full*) ou se está vazia (*Empty*). O bloco *Ring Buffer* procede à entrega de dados à entidade consumidora, sempre que esta indique que está disponível para receber, através do sinal *Clear To Send* (CTS).

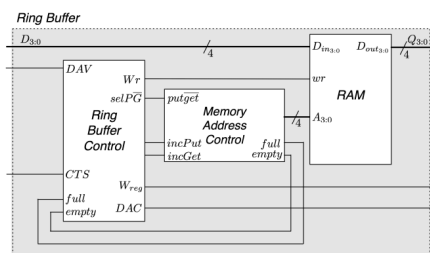


Figura 5: Diagrama de blocos do *RingBuffer*

2.1 Memory Address Control (MAC)

O bloco MAC foi projetado e implementado para indicar o endereço respetivo à operação realizada de acordo com a entrada de seleção PnG do bloco. O endereço respetivo a cada operação realizada pelo bloco está dividido por "put" e "get", que armazenam o respetivo endereço através do seu respetivo contador de 5 bits. Estes contadores são responsáveis por guardar o endereço disponível para escrita ou leitura na RAM e sempre que utilizados, selecionam o seguinte para futura utilização a partir das entradas *incPut* e *incGet*, que ligam à entrada "CE" (Clock Enable) dos contadores, permitindo-lhes realizar a incrementação de uma posição. O endereço que é efetivamente acedido pela RAM é disponibilizado na saída S de 4 bits. A seleção de qual endereço fornecer é feita através de uma atribuição condicional: o bloco seleciona os 4 bits menos significativos (3 down to 0) do contador de leitura se o sinal PnG for '0', ou os 4 bits do contador de escrita caso contrário. Desta maneira mantemos a organização dos mesmos de acordo com a disciplina da estrutura de endereços FIFO (*First In First Out*). Para evitar perda de dados por escrita em endereços que não foram lidos ou ler endereços que já foram lidos sem nova escrita, adotou-se a estratégia de utilizar o bit mais significativo dos contadores como um marcador de ciclo. Sabendo que temos 16 posições de endereçamento físico, a arquitetura implementa contadores de 5 bits, onde o 5º bit (o bit na posição 4) funciona como um indicador para saber quantas vezes o ponteiro respetivo completou uma volta inteira à memória. A ativação da flag *full* foi feita a partir da comparação entre as saídas dos dois contadores: esta flag é ativada quando os 4 bits menos significativos de ambos os ponteiros são iguais, mas os seus 5ºs bits (bit 4) são diferentes. Esta condição sinaliza que o ponteiro de escrita deu exatamente uma volta completa a mais que o de leitura, alcançando-o por trás, o que corresponde ao estado de ocupação máxima do buffer. A mesma estratégia foi adotada para a flag *empty*, onde é realizada uma comparação em que esta é ativada quando ambos os contadores apresentam valores exatamente iguais, ou seja, os 4 bits de endereçamento físico coincidem e o 5º bit também é idêntico. Isto significa que o ponteiro de leitura alcançou o de escrita, representando o

estado vazio do Ring Buffer.

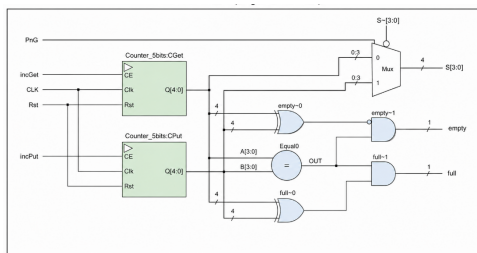


Figura 6: Diagrama de blocos do Memory Address Control (*MAC*)

2.2 Ring Buffer Control

Este bloco foi implementado a partir de uma máquina de estados obtida a partir da interpretação do funcionamento do sistema discriminado no enunciado e da interpretação do funcionamento dos outros blocos que estão contidos no Ring Buffer. Como podemos observar na figura 7, a máquina de estados é composta por 7 estados, sendo o primeiro "Idle" que é o estado inicial onde todos os valores das saídas do têm o valor lógico de '0'. Depois do estado "Idle" realizamos verificações a partir dos valores lógicos presentes nas entradas pois estes vão terminar a operação que o bloco irá realizar: Writing ou Reading. Caso a entrada DAV esteja ativa e a flag full desativada o bloco irá realizar a operação Writing que consiste em uma sequência de 4 estados, onde no primeiro estado (prev_write) a entrada PnG é ativa, de seguida passamos para o segundo estado (Writing) onde a entrada PnG continua ativa e ativamos a saída Wr, no terceiro estado (pos_write) ativamos somente a entrada ininput e passamos para o último estado (ack_producer) que ativa a saída DAC e só volta para o estado inicial (Idle) quando a entrada DAV estiver a zero, caso contrário estará em um loop nesse mesmo estado, no caso da flag full se encontrar ativa vai verificar o estado do CTS que se estiver ativo seguirá para o estado de Reading, caso contrário voltará ao estado de Idle. Já no caso de Read temos a verificação de CTS com o DAV a '0', que deverá estar ativa e de Empty que deverá estar desativada. O primeiro estado "Reading" consiste na ativação da saída Wreg e verificação da entrada

CTS que deverá estar desativada para passarmos para o segundo estado "pos_Read" que ativa a saída incget e volta ao estado Idle. A máquina de estados anteriormente descrita foi traduzida para código VHDL no ficheiro do Ring Buffer Control.

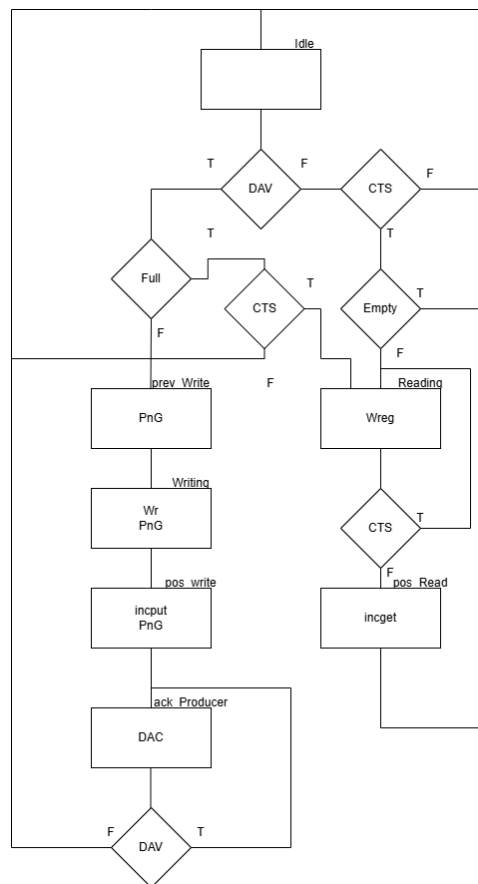


Figura 7: Máquina de estados *Ring Buffer Control*

2.3 RAM

Bloco responsável por armazenar os bits provenientes da entrada $D_{in3,0}$ no endereço fornecido pelo bloco MAC e fornecer os dados armazenados através da saída $D_{out3,0}$. Este bloco foi fornecido pelos docentes da disciplina na página geral do moodle <Laboratório de Informática e Computadores- LEIC- 2526SV>.

3 Key Transmitter

O bloco Key Transmitter desempenha um papel fundamental no subsistema de leitura do teclado (Keyboard Reader) da Ticket Machine. A sua principal responsabilidade é atuar como uma interface de comunicação série entre o módulo produtor (responsável por registar fisicamente a tecla pressionada) e o módulo consumidor (o módulo Control central do sistema).

Este bloco foi desenhado com a capacidade de armazenar uma palavra de quatro bits em paralelo e transmiti-la de forma sequencial. A comunicação é estritamente regida pelo protocolo ilustrado na figura seguinte e sincronizada por um sinal de relógio próprio (TXclk).

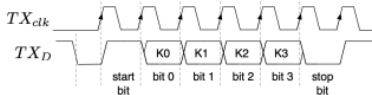


Figura 8: Protocolo de comunicação com o módulo Key Transmitter

Para garantir a fiabilidade na transferência de dados, é utilizado um mecanismo de handshaking. O transmissor sinaliza a sua disponibilidade para armazenar uma nova tecla ativando o sinal KBfree. Quando o módulo produtor tem dados válidos, ativa o sinal Load para os guardar na memória interna do transmissor. Concluída a escrita, o transmissor desativa o KBfree e inicia a transmissão, enviando na linha TXD uma trama composta por um start bit (nível baixo), seguido dos 4 bits de dados e terminando com um stop bit (nível alto).

3.1 DataPath

O módulo KT_Datath aloj os componentes físicos necessários para armazenar e encaminhar os bits corretos para a saída série. A sua estrutura interna é composta por três submódulos:

Registo (KT_Register): Responsável por guardar a palavra de 4 bits (K) vinda do exterior. A gravação ocorre apenas quando o sistema está no estado livre (KBfree ativo) e recebe a indicação de escrita (Load ativo).

Contador (KT_Counter): Um contador sincronizado pelo relógio de transmissão (TXclk) e ati-

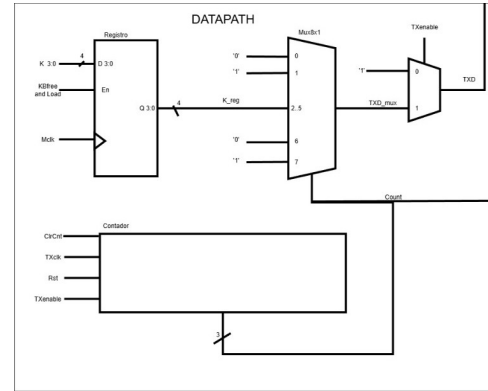


Figura 9: Diagrama de blocos do módulo Key Transmitter

vado pelo sinal de controlo TXEnable. O seu valor de saída (Count) atua como um índice que dita que fase da transmissão está a decorrer.

Mux (KT_Mux8x1): Atua como seletor da trama de dados. Dependendo do valor do Count, este encaminha para a saída sequencialmente: o start bit ('0'), start bit ('1'), os bits de dados armazenados no registo (K_0 a K_3), stop bit ('1') e o stop bit ('0').

A saída final TXD é decidida condicionalmente: se o sistema sinalizar o arranque (StartTX = '1'), a saída é forçada a '0'. Quando a transmissão está a decorrer (TXEnable = '1'), a saída reflete o valor do mux. Quando inativo, a linha mantém-se em Idle ('1').

3.2 Control KT

O módulo KT_Control atua como o "cérebro" do transmissor, coordena o caminho de dados. O comportamento desta unidade foi desenhado com base no diagrama ASM fornecido, materializado através de uma Máquina de Estados Finita (FSM) descrita de forma comportamental no VHDL.

A FSM é composta por 2 estados essenciais que traduzem o fluxo do fluxograma:

Free (Espera): É o estado de repouso (equivalente ao bloco KBfree no fluxograma). Neste estado, a saída KBfree e ClrCnt são ativadas ('1'). A máquina avalia a condição de entrada Load. Permanece no estado Free caso seja Falso ('0'), mas transita para o estado seguinte caso o Load passe

a Verdadeiro ('1').

Transmit (Transmissão): Este estado gere o envio do corpo principal da mensagem. A saída TX-Enable é colocada a '1', permitindo que o contador avance e o multiplexador percorra os dados. A transição para fora deste estado é gerida pela condição de paragem: a FSM monitoriza o sinal Count e, assim que este atinge o valor lógico de fim ("111"), a transmissão é dada como concluída. A máquina regressa ao estado Free, pronta para processar a próxima tecla.

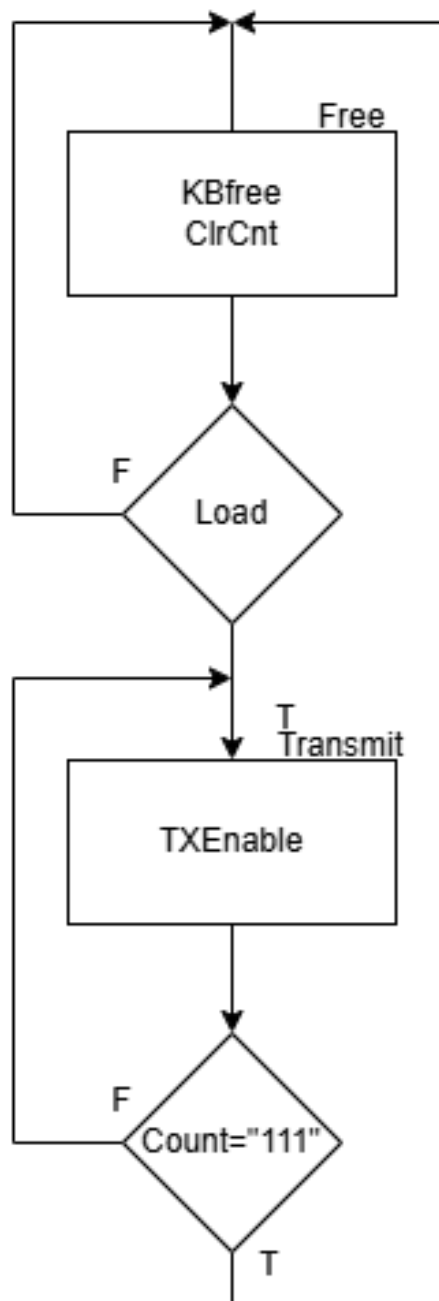


Figura 10: Máquina de estados do módulo KT_Control

4 Conclusões

O módulo Keyboard Reader demonstrou ser um componente essencial na arquitetura do Ticket Machine, atuando com sucesso como a interface responsável por decodificar as interações com o teclado matricial de 16 teclas e disponibilizar o respetivo código de quatro bits ao módulo central (Control). A arquitetura dividiu o problema em três blocos principais: O bloco Key Decode cumpriu a sua função de decodificador de hardware do teclado matricial 4x4, garantindo a deteção e o varrimento das teclas. A inclusão do bloco Ring Buffer foi crucial para a tolerância a atrasos no processamento, permitindo o armazenamento de dados através de uma disciplina FIFO (First In First Out) com capacidade para 16 palavras de quatro bits. Isto garante que a informação não se perde caso o módulo Control não esteja imediatamente disponível para receber os dados. Por fim, o bloco Key Transmitter demonstrou eficácia na entrega dos dados à entidade consumidora, implementando uma estrutura de transmissão série para enviar os códigos armazenados para o módulo Control.

A VHDL

A.1 KeyBoard Reader

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyboardReader is
port(
    MClk      : in std_logic;
    Rst       : in std_logic;
    Tdelay    : in std_logic_vector(1 downto 0);
    linhas_in : in std_logic_vector(3 downto 0);
    Txclk     : in std_logic;
    K         : out std_logic_vector(3 downto 0);
    colunas_out : out std_logic_vector(3 downto 0);
    TxD       : out std_logic
);
end KeyboardReader;

architecture structural of KeyboardReader is

    component KeyDecode is
        port(
            MClk      : in std_logic;
            Rst       : in std_logic;
            Tdelay    : in std_logic_vector(1 downto 0);
            Ka        : in std_logic;
            linhas_in : in std_logic_vector(3 downto 0);
            colunas_out : out std_logic_vector(3 downto 0);
            K         : out std_logic_vector(3 downto 0);
            Kval      : out std_logic
        );
    end component;

    component RingBuffer is
        port(
            CLK      : in std_logic;
            RESET    : in std_logic;
            DAV      : in std_logic;
            CTS      : in std_logic;
            Din       : in std_logic_vector(3 downto 0);
            DAC       : out std_logic;
            Wreg      : out std_logic;
            Dout      : out std_logic_vector(3 downto 0)
        );
    end component;

    component KeyTransmitter is
        port(
            MClk      : in std_logic;
            K         : in std_logic_vector(3 downto 0);
            Load      : in std_logic;
            TXclk     : in std_logic;
            Rst       : in std_logic;
            K_out      : out std_logic_vector(3 downto 0);
            KBfree     : out std_logic;
            TXD       : out std_logic
        );
    end component;

    signal Kval_out : std_logic;
    signal KRB_out, dataTx : std_logic_vector(3 downto 0);
    signal Ka_in      : std_logic;
```

```
signal Q_out      : std_logic_vector(3 downto 0);
signal Kf_out     : std_logic;
signal W_out      : std_logic;
```

```
begin
```

```
KeyDecode_V : KeyDecode
```

```
  port map(
    MClk      => MClk,
    Rst       => Rst,
    Tdelay    => Tdelay,
    Ka        => Ka_in,
    linhas_in => linhas_in,
    colunas_out => colunas_out,
    K         => KRB_out,
    Kval      => Kval_out
  );
```

```
RingBuffer_V : RingBuffer
```

```
  port map(
    CLK  => MClk,
    RESET => Rst,
    DAV  => Kval_out,
    CTS  => Kf_out,
    Din  => KRB_out,
    DAC  => Ka_in,
    Wreg => W_out,
    Dout => Q_out
  );
```

```
KeyTransmitter_V : KeyTransmitter
```

```
  port map(
    MClk  => MClk,
    K      => Q_out,
    Load => W_out,
    TXclk  => TxClk,
           K_out      => DataTx,
    Rst    => Rst,
    KBfree => Kf_out,
    TXD    => TxD
  );
```

```
    K <= DataTX;
```

```
end structural;
```

A.2 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyDecode is
  port(
    MClk      : in std_logic;
    Rst       : in std_logic;
    Tdelay    : in std_logic_vector(1 downto 0);
    Ka        : in std_logic;
    linhas_in : in std_logic_vector(3 downto 0);
    colunas_out : out std_logic_vector(3 downto 0);
    K         : out std_logic_vector(3 downto 0);
    Kval      : out std_logic
  );
```

```
end KeyDecode;

architecture structural of KeyDecode is

component KeyScan is
    port(
        MClk      : in std_logic;
        Kscan     : in std_logic;
        Rst       : in std_logic;
        linhas_in  : in std_logic_vector(3 downto 0);
        colunas_out : out std_logic_vector(3 downto 0);
        K         : out std_logic_vector(3 downto 0);
        Kpress    : out std_logic
    );
end component;

component KeyControl is
    port(
        MClk  : in std_logic;
        Rst   : in std_logic;
        Ka    : in std_logic;
        Tdelay : in std_logic_vector(1 downto 0);
        Kpress : in std_logic;
        Kval   : out std_logic;
        Kscan  : out std_logic
    );
end component;

signal s_Kscan : std_logic;
signal s_Kpress : std_logic;
signal s_clk : std_logic;

begin

KeyScan_V: KeyScan
    port map(
        MClk      => MClk,
        Kscan     => s_Kscan,
        Rst       => Rst,
        linhas_in  => linhas_in,
        colunas_out => colunas_out,
        K         => K,
        Kpress    => s_Kpress
    );

KeyColtrol_V: KeyControl
    port map(
        MClk  => MClk,
        Rst   => Rst,
        Ka    => Ka,
        Tdelay => Tdelay,
        Kpress => s_Kpress,
        Kval   => Kval,
        Kscan  => s_Kscan
    );

end structural;
```

A.3 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyControl is
port
    (
        MClk : in std_logic;
        Rst   : in std_logic;
        Ka    : in std_logic;
        Tdelay : in std_logic_vector(1 downto 0);
        Kpress: in std_logic;
        Kval  : out std_logic;
        Kscan : out std_logic
    );
end KeyControl;

architecture behavioral of KeyControl is

type State_Type is ( Scan, Delievery, NotAcknowledge, KeyRelease);

signal CurrentState, NextState: State_Type;

begin

CurrentState <= Scan when Rst = '1' else NextState when rising_edge(MClk);

GenerateNextState:
process ( CurrentState, Ka, Kpress)
begin
    case CurrentState is
        when Scan => if (Kpress = '1')
                        then
                            NextState <= Delievery;
                        else
                            NextState <= Scan;
                        end if;

        when Delievery => if (Ka = '1')
                            then
                                NextState <= NotAcknowledge;
                            else
                                NextState <= Delievery;
                            end if;

        when NotAcknowledge => if (KA = '0')
```

```

                                then
                                    NextState <= KeyRelease;
                                else
                                    NextState <= NotAcknowledge;
                                end if;

                                when KeyRelease => if (Kpress = '0')
                                    then
                                        NextState <= Scan;
                                    else
                                        NextState <= KeyRelease;
                                    end if;

                                end case;
                            end process;

Kval <= '1' when ( (CurrentState = Delievery) ) else '0';
Kscan <= '1' when ( (CurrentState = Scan) ) else '0';

end behavioral;

```

A.4 Key Scan

```

library ieee;
use ieee.std_logic_1164.all;

entity KeyScan is
port(
    MClk      : in std_logic;
    Kscan     : in std_logic;
    Rst       : in std_logic;
    linhas_in : in std_logic_vector(3 downto 0);
    colunas_out : out std_logic_vector(3 downto 0);
    K         : out std_logic_vector(3 downto 0);
    Kpress    : out std_logic
);
end KeyScan;

architecture structural of KeyScan is

    component Counter is
        port(
            Clk : in std_logic;
            CE  : in std_logic;
            Rst : in std_logic;
            Q   : out std_logic_vector(3 downto 0)
        );
    end component;

    component TM_Decoder is
        port(
            S0: in std_logic;
            S1: in std_logic;
            C0: out std_logic;
            C1: out std_logic;
            C2: out std_logic;
            C3: out std_logic
        );
    end component;

    component Mux4x1 is

```

```

port(
    A : in std_logic;
        B : in std_logic;
        C : in std_logic;
        D : in std_logic;
        S : in std_logic_vector(1 downto 0);
        Y : out std_logic
    );

end component;

signal CE_sig : std_logic;
signal QS      : std_logic_vector(3 downto 0);
signal NotCols: std_logic_vector(3 downto 0);
signal Not_Y: std_logic;
signal clkOut: std_logic;

begin

CE_sig <= Kscan;

Cont : Counter
port map(
    Clk => MClk,
    CE  => CE_sig,
    Rst => Rst,
    Q   => QS
);

Dec : TM_Decoder
port map(
    S0 => QS(2),
    S1 => QS(3),
    C0 => NotCols(0),
    C1 => NotCols(1),
    C2 => NotCols(2),
    C3 => NotCols(3)
);

MUX : Mux4x1
port map(
    S(0) => QS(0),
    S(1) => QS(1),
        A   => linhas_in(0),
        B   => linhas_in(1),
        C   => linhas_in(2),
        D   => linhas_in(3),
    Y     => Not_Y
);

K <= QS;
colunas_out <= not NotCols;
Kpress <= not Not_Y;

end structural;

```

A.5 Ring Buffer

```

library ieee;
use ieee.std_logic_1164.all;

entity RingBuffer is

```

```
port(
    CLK    : in  std_logic;
    RESET  : in  std_logic;
    DAV     : in  std_logic;
    CTS     : in  std_logic;
    Din     : in  std_logic_vector(3 downto 0);

    DAC     : out std_logic;
    Wreg    : out std_logic;
    Dout    : out std_logic_vector(3 downto 0)
);
end RingBuffer;

architecture structural of RingBuffer is

    component RingBufferControl is
        port(
            CLK    : in  std_logic;
            RESET  : in  std_logic;
            DAV     : in  std_logic;
            CTS     : in  std_logic;
            full    : in  std_logic;
            empty   : in  std_logic;

            Wr      : out std_logic;
            PnG     : out std_logic;
            incPut  : out std_logic;
            incGet  : out std_logic;
            Wreg    : out std_logic;
            DAC     : out std_logic
        );
    end component;

    component MAC is
        port(
            CLK    : in  std_logic;
            Rst    : in  std_logic;
            PnG    : in  std_logic;
            incPut  : in  std_logic;
            incGet  : in  std_logic;
            S      : out std_logic_vector(3 downto 0);
            full    : out std_logic;
            empty   : out std_logic
        );
    end component;

    component RAM is
        generic(
            ADDRESS_WIDTH : natural := 4;
            DATA_WIDTH    : natural := 4
        );
        port(
            address : in  std_logic_vector(ADDRESS_WIDTH - 1 downto 0);
            wr      : in  std_logic;
            din     : in  std_logic_vector(DATA_WIDTH - 1 downto 0);
            dout    : out std_logic_vector(DATA_WIDTH - 1 downto 0)
        );
    end component;

    signal Wr_s      : std_logic;
    signal PnG_s     : std_logic;
    signal incPut_s   : std_logic;
    signal incGet_s   : std_logic;
    signal Wreg_s    : std_logic;
```

```

signal A_s      : std_logic_vector(3 downto 0);
signal full_s   : std_logic;
signal empty_s  : std_logic;

signal Dout_ram : std_logic_vector(3 downto 0);

begin

    CONTROL : RingBufferControl
        port map(
            CLK      => CLK,
            RESET    => RESET,
            DAV      => DAV,
            CTS      => CTS,
            full     => full_s,
            empty    => empty_s,
            Wr       => Wr_s,
            PnG      => PnG_s,
            incPut   => incPut_s,
            incGet   => incGet_s,
            Wreg     => Wreg_s,
            DAC      => DAC
        );

    ADDRESS_CONTROL : MAC
        port map(
            CLK      => CLK,
            Rst      => RESET,
            PnG      => PnG_s,
            incPut   => incPut_s,
            incGet   => incGet_s,
            S        => A_s,
            full     => full_s,
            empty    => empty_s
        );

    MEMORY : RAM
        generic map(
            ADDRESS_WIDTH => 4,
            DATA_WIDTH   => 4
        )
        port map(
            address => A_s,
            wr      => Wr_s,
            din     => Din,
            dout    => Dout_ram
        );

    Dout <= Dout_ram;
    Wreg <= Wreg_s;

end structural;

```

A.6 Ring Buffer Control

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY RingBufferControl IS
    PORT(
        CLK      : in std_logic;
        RESET    : in std_logic;

```

```
    DAV    : in std_logic;
    CTS    : in std_logic;
    full   : in std_logic;
    empty  : in std_logic;

    Wr      : out std_logic;
    PnG     : out std_logic;
    incPut  : out std_logic;
    incGet  : out std_logic;
    Wreg    : out std_logic;
    DAC     : out std_logic
);
END RingBufferControl;

architecture Behavioral of RingBufferControl is
--    type state_type is (Idle, prev_Write, Writing, pos_Write, Reading, pos_Read, ack_producer);
    type state_type is (Idle, prev_Write, Writing, pos_Write, Reading, pos_Read, ack_producer);
    signal current_state, next_state : STATE_TYPE;

begin

    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= Idle;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    process(current_state, DAV, CTS, Full, Empty)
    begin
--
--        Wr <= '0';
--        incPut <= '0';
--        incGet <= '0';
--        PnG <= '0';
--        DAC <= '0';
--        Wreg <= '0';

        case current_state is

            when Idle =>

                if DAV = '1' and Full = '0' then
                    next_state <= prev_Write;

                elsif DAV = '1' and Full = '1' and CTS = '1' then
                    next_state <= Reading;

                elsif DAV = '0' and CTS = '1' and Empty = '0' then
                    next_state <= Reading;

                else
                    next_state <= Idle;
                end if;

                when prev_Write =>

                    next_state <= Writing;

            end case;

        end process;

    end architecture;
```

```

when Writing =>

    next_state <= pos_Write;

when pos_Write =>
    next_state <= ack_producer;

    when ack_producer =>

        if DAV = '1' then
            next_state <= ack_producer;
        else
            next_state <= Idle;
        end if;

    when Reading =>

        if CTS = '1' then
            next_state <= Reading;
        else
            next_state <= pos_Read;
        end if;

    when pos_Read =>

        next_state <= Idle;

end case;
end process;

Wr <= '1' when (current_state = Writing) else '0' ;

incPut <= '1' when (current_state = pos_Write) else '0' ;

incGet <= '1' when (current_state = pos_Read) else '0' ;

PnG <= '1' when (current_state = prev_Write) or (current_state = Writing) or (current_state = pos_Write) else '0' ;

DAC <= '1' when (current_state = ack_producer) else '0' ;

Wreg <= '1' when (current_state = Reading) else '0';

end Behavioral;

```

A.7 Memory Address Control (MAC)

```

library ieee;
use ieee.std_logic_1164.all;

entity MAC is
    port(
        CLK      : in  std_logic;
        Rst       : in  std_logic;
        PnG       : in  std_logic;
        incPut     : in  std_logic;
        incGet     : in  std_logic;
        S         : out std_logic_vector(3 downto 0);
        full       : out std_logic;
        empty      : out std_logic
    );
end entity;

```

```

    );
end MAC;

architecture structural of MAC is

    component Counter_5bits is
        port(
            Clk : in std_logic;
            CE  : in std_logic;
            Rst : in std_logic;
            Q   : out std_logic_vector(4 downto 0)
        );
    end component;

    signal Q_out_CPut : std_logic_vector(4 downto 0);
    signal Q_out_CGet : std_logic_vector(4 downto 0);

begin

    CPut : Counter_5bits
        port map(
            Clk => CLK,
            CE  => incPut,
            Rst => Rst,
            Q   => Q_out_CPut
        );

    CGet : Counter_5bits
        port map(
            Clk => CLK,
            CE  => incGet,
            Rst => Rst,
            Q   => Q_out_CGet
        );

    S <= Q_out_CGet(3 downto 0) when PnG = '0' else
        Q_out_CPut(3 downto 0);

    full <= '1' when (Q_out_CPut(3 downto 0) = Q_out_CGet(3 downto 0)) and
        (Q_out_CPut(4) /= Q_out_CGet(4))
        else '0';

    empty <= '1' when (Q_out_CPut(3 downto 0) = Q_out_CGet(3 downto 0)) and
        (Q_out_CPut(4) = Q_out_CGet(4))
        else '0';

end structural;

```

A.8 Key Transmitter

```

library ieee;
use ieee.std_logic_1164.all;

entity KeyTransmitter is
    port(
        MClk   : in std_logic;
        K       : in std_logic_vector(3 downto 0);
        Load   : in std_logic;
        TXclk   : in std_logic;
        Rst     : in std_logic;
        KBfree  : out std_logic;
        K_out   : out std_logic_vector(3 downto 0);
        TXD     : out std_logic
    );
end entity;

```

```

    );
end KeyTransmitter;

architecture structural of KeyTransmitter is

    component KT_Control is
        port(
            MClk      : in  std_logic;
            Rst       : in  std_logic;
            Load      : in  std_logic;
            TXclk     : in  std_logic;
            Count     : in  std_logic_vector(2 downto 0);
            KBfree    : out std_logic;
            ClrCnt    : out std_logic;
            TXEnable  : out std_logic
        );
    end component;

    component KT_Datapath is
        port(
            MClk      : in  std_logic;
            TXclk     : in  std_logic;
            K         : in  std_logic_vector(3 downto 0);
                        KBfree    : in  std_logic;
            Load      : in  std_logic;
            TXEnable  : in  std_logic;
            ClrCnt    : in  std_logic;
            Rst       : in  std_logic;
                        K_out     : out std_logic_vector(3 downto 0);
            Count     : out std_logic_vector(2 downto 0);
            TXD      : out std_logic
        );
    end component;

    signal Count_s      : std_logic_vector(2 downto 0);
    signal KBfree_s     : std_logic;
    signal TXEnable_s   : std_logic;
    signal ClrCnt_s     : std_logic;

begin

    Control1 : KT_Control
        port map(
            MClk      => MClk,
            Rst       => Rst,
            Load      => Load,
            TXclk     => TXclk,
            Count     => Count_s,
            KBfree    => KBfree_s,
            ClrCnt    => ClrCnt_s,
            TXEnable  => TXEnable_s
        );

    Datapath1 : KT_Datapath
        port map(
            MClk      => MClk,
            TXclk     => TXclk,
            K         => K,
                        KBfree    => KBfree_s,
            Load      => Load,
            TXEnable  => TXEnable_s,
            ClrCnt    => ClrCnt_s,
            Rst       => Rst,

```

```

        Count    => Count_s,
        TXD      => TXD
    );

    KBfree <= KBfree_s;
end structural;

```

A.9 KT_Datapath

```

library ieee;
use ieee.std_logic_1164.all;

entity KT_Datapath is
    port(
        MClk      : in std_logic;
        TXclk      : in std_logic;
        K          : in std_logic_vector(3 downto 0);
        KBfree     : in std_logic;
        Load       : in std_logic;
        TXEnable   : in std_logic;
        ClrCnt     : in std_logic;
        Rst        : in std_logic;
        K_out      : out std_logic_vector(3 downto 0);
        Count      : out std_logic_vector(2 downto 0);
        TXD        : out std_logic
    );
end KT_Datapath;

architecture structural of KT_Datapath is

    component KT_Register is
        port(
            MClk : in std_logic;
            Rst  : in std_logic;
            K    : in std_logic_vector(3 downto 0);
            KBfree : in std_logic;
            Load : in std_logic;
            K_reg : out std_logic_vector(3 downto 0)
        );
    end component;

    component KT_Counter is
        port(
            TXclk : in std_logic;
            TXEnable : in std_logic;
            ClrCnt : in std_logic;
            Rst    : in std_logic;
            Q      : out std_logic_vector(2 downto 0)
        );
    end component;

    component KT_Mux8x1 is
        port(
            Start0 : in std_logic;
            Start1 : in std_logic;
            K       : in std_logic_vector(3 downto 0);
            Stop0   : in std_logic;
            Stop1   : in std_logic;
            Selector : in std_logic_vector(2 downto 0);
            Y       : out std_logic
        );
    end component;

```

```

end component;

signal K_reg_s : std_logic_vector(3 downto 0);
signal Count_s : std_logic_vector(2 downto 0);
signal TXD_mux : std_logic;

begin

    Reg1 : KT_Register
        port map(
            MClk    => MClk,
            Rst     => Rst,
            K       => K,
            KBfree  => KBfree,
            Load    => Load,
            K_reg   => K_reg_s
        );

    Counter1 : KT_Counter
        port map(
            TXclk    => TXclk,
            TXEnable => TXEnable,
            ClrCnt   => ClrCnt,
            Rst      => Rst,
            Q        => Count_s
        );

    DataMux : KT_Mux8x1
        port map(
            Start0  => '0',
            Start1  => '1',
            K       => K_reg_s,
            Stop0   => '0',
            Stop1   => '1',
            Selector => Count_s,
            Y       => TXD_mux
        );

    Count <= Count_s;

    TXD <= TXD_mux when TXEnable = '1' else '1';

    K_out <= K_reg_s;

end structural;

```

A.10 KT_Control

```

library ieee;
use ieee.std_logic_1164.all;

entity KT_Control is
port(
    MClk    : in std_logic;
    TXclk   : in std_logic;
    Rst     : in std_logic;
    Load   : in std_logic;
    Count   : in std_logic_vector(2 downto 0);

    KBfree  : out std_logic;
    ClrCnt  : out std_logic;
    TXEnable : out std_logic

```

```
);  
end KT_Control;  
  
architecture behavioral of KT_Control is  
  
    type State_Type is (  
        Free,  
        Transmit  
    );  
  
    signal CurrentState, NextState : State_Type;  
  
begin  
  
    CurrentState <= Free when Rst = '1' else NextState when rising_edge(MClk);  
  
    GenerateNextState:  
    process(CurrentState, Load, TXclk, Count)  
    begin  
  
        case CurrentState is  
  
            when Free =>  
                if Load = '1' then  
                    NextState <= Transmit;  
                else  
                    NextState <= Free;  
                end if;  
  
            when Transmit =>  
                if Count = "111" and TXclk = '0' then  
                    NextState <= Free;  
                else  
                    NextState <= Transmit;  
                end if;  
  
        end case;  
  
    end process;  
  
    KBfree <= '1' when CurrentState = Free else '0';  
  
    ClrCnt <= '1' when CurrentState = Free else '0';  
  
    TXEnable <= '1' when CurrentState = Transmit else '0';  
  
end behavioral;
```

B *Kotlin*

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
import isel.leic.UsbPort
import isel.leic.utils.Time

object HAL {
    private var outPort = 0

    fun init() {
        outPort = 0
        UsbPort.write(outPort)
    }

    fun readBits(mask: Int): Int {
        return UsbPort.read() and mask
    }

    fun isBit(mask: Int): Boolean {
        return (UsbPort.read() and mask) != 0
    }

    fun setBits(mask: Int) {
        outPort = outPort or mask
        UsbPort.write(outPort)
    }

    fun clrBits(mask: Int) {
        outPort = outPort and mask.inv()
        UsbPort.write(outPort)
    }

    fun writeBits(mask: Int, value: Int) {
        outPort = (outPort and mask.inv()) or (value and mask)
        UsbPort.write(outPort)
    }
}

//fun main(Args: Array<String>){
//    while(true){
//        //    val value = UsbPort.read()
//        //    UsbPort.write(value)
//        println(HAL.readBits(0xFF))
//    }
//}
```

B.2 KBD

Algoritmo 2: KBD

```
object KBD {
    const val NONE = 0
    val none = NONE.toChar()

    fun init() {
        HAL.init()
        SerialReceiver.init()
    }

    fun getKey(): Char {
        val keyCode = SerialReceiver.rcv()
        if (keyCode == INVALID_KEYCODE) return none

        return when (keyCode) {
            0 -> '1'
            1 -> '4'
        }
    }
}
```

```

        2 -> '7'
        3 -> '*'
        4 -> '2'
        5 -> '5'
        6 -> '8'
        7 -> '0'
        8 -> '3'
        9 -> '6'
        10 -> '9'
        11 -> '#'
        12 -> 'A'
        13 -> 'B'
        14 -> 'C'
        15 -> 'D'
        else -> none
    }
}

fun waitKey(timeout: Long): Char {
    val start = System.currentTimeMillis()

    while (System.currentTimeMillis() - start < timeout) {
        val key = getKey()
        if (key != none) return key
    }

    return none
}

fun main() {
    KBD.init()
    while (true) {
        val k = KBD.getKey()
        if (k != KBD.none) {
            println("tecla=${k}")
        }
    }
}

```